KIGALI INDEPENDENT UNIVERSITY (ULK)



UML CLASS DIAGRAM MODELLING ASSIGNMENT

UML CLASS DIAGRAM REPORT

Project Title: Online Shopping System
Instructor: Eric Maniraguha
Group Members

1. **Student 1 : 202510585**
2. **Student 2 : 202512742**

Submission Date: 20 October 2026

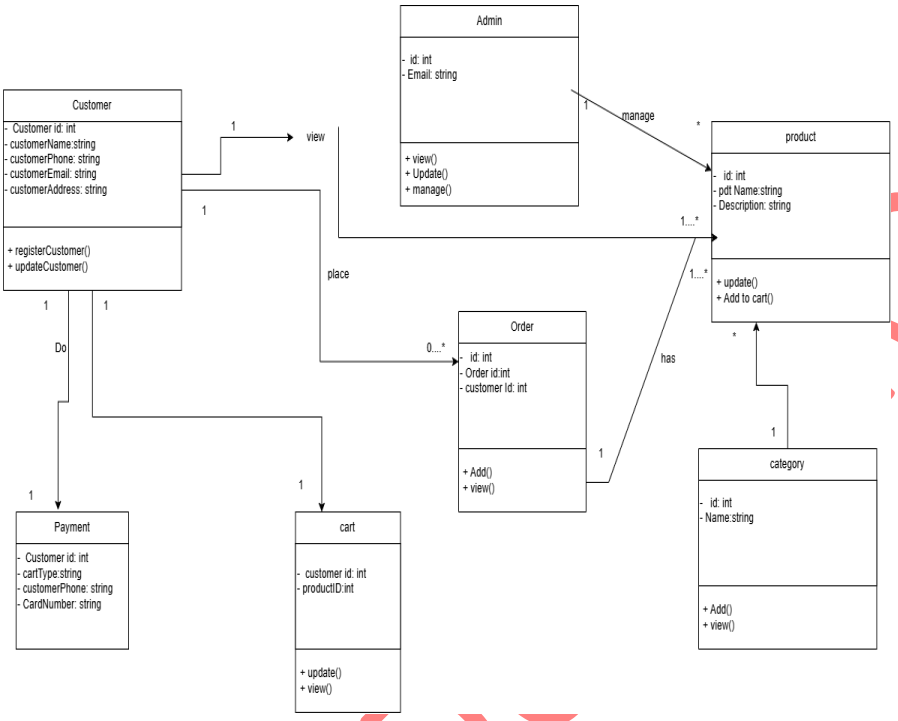
SQL MODULE YR2 CSDS

1. Introduction

A UML (Unified Modeling Language) class diagram is a structural diagram used to represent the classes of a system, their attributes, their operations, and the relationships between them.

This report describes the UML class diagram designed for the proposed Online Shopping System. The diagram contains seven classes: Admin, Customer, Product, Category, Cart, Order, and Payment.

DIAGRAM



2. Purpose of the Class Diagram

The purpose of the diagram is to represent the static structure of the proposed system. It identifies the main entities involved in online shopping and shows how they are connected.

The diagram helps to clarify the responsibilities of the different classes and provides a basis for future system development.

3. Description of the Classes

SOLMOD 12 CSDS

3.1 Admin Class

Purpose: Represents an administrator who manages products in the system.

Attributes:

- id: int
- Email: string

Operations shown in the diagram:

- view()
- Update()
- manage()

The Admin class represents administrative activities. The diagram associates the administrator with product management.

3.2 Customer Class

Purpose: Represents a customer who uses the online shopping system.

Attributes:

- Customer id: int
- customerName: string
- customerPhone: string
- customerEmail: string
- customerAddress: string

Operations:

- registerCustomer()
- updateCustomer()

The Customer class stores customer details and represents the customer activities of registration and updating information.

3.3 Product Class

Purpose: Represents products available in the online shop.

Attributes:

- id: int
- pdt Name: string

SOL MODULE YR2 CSDS

- Description: string

Operations:

- update()
- Add to cart()

The Product class stores basic product information. The diagram associates products with administrator management, customer viewing, shopping cart activities, and categories.

3.4 Category Class

Purpose: Represents categories used to organise products.

Attributes:

- id: int
- Name: string

Operations:

- Add()
- view()

The Category class provides a way to represent product groupings within the online shopping system.

3.5 Cart Class

Purpose: Represents the shopping cart associated with customer shopping activities.

Attributes:

- customer id: int
- productID: int

Operations:

- update()
- view()

The Cart class represents the association between a customer and products selected for shopping. The exact multiplicity and whether one cart can contain multiple products should be made clear in the final diagram.

SQL MODULE YR2 CSDS

3.6 Order Class

Purpose: Represents an order placed by a customer.

Attributes:

- id: int
- Order id: int
- customer Id: int

Operations:

- Add()
- view()

The Order class represents the ordering activity in the proposed system. The diagram associates Customer with Order to show that customers place order

3.7 Payment Class

Purpose: Represents payment-related information in the proposed system.

Attributes:

- Customer id: int
- cartType: string
- customerPhone: string
- CardNumber: string

Operations:

- update()
- view()

The Payment class represents payment-related information connected to the shopping process. The relationship between Order and Payment should be clearly indicated in the final diagram.

Security consideration: In a real payment system, full card numbers should not be stored as ordinary plain-text data. A secure payment provider or tokenised payment method should be used. This is a design consideration and does not imply that the current diagram implements payment security.

4. Relationships Between Classes

SOL MODULE YR2 CSDS

The following relationships are represented or suggested by the submitted class diagram.

4.1 Admin and Product

The diagram shows that Admin manages Product.

This relationship represents the administrator's responsibility for managing product information. The multiplicity shown in the drawing appears to indicate that one administrator can manage multiple products.

4.2 Customer and Product

The diagram shows that Customer can view Product.

This association represents a customer viewing products available in the online shop. Viewing a product is an interaction between the customer and the product catalogue.

4.3 Customer and Order

The diagram shows that Customer places Order.

The multiplicity in the drawing appears to indicate that one customer can place zero or many orders, represented as 0..* at the Order end. The customer end should be checked to confirm the intended multiplicity.

4.4 Customer and Cart

The diagram connects Customer with Cart.

This relationship represents a customer's association with a shopping cart. The intended multiplicity should be made explicit and consistent with the business rules.

4.5 Cart and Product

The Cart class contains a product identifier, and the diagram represents a connection between shopping cart activities and products.

The final diagram should clearly indicate whether one cart can contain multiple products and whether a product can appear in many different carts.

4.6 Category and Product

The diagram connects Category with Product.

SOL MODULE YR2 CSDS

This relationship represents the organisation of products into categories. The multiplicities should be shown clearly at both ends of the association.

4.7 Order and Payment

The diagram shows a relationship labelled as a payment-related association between Order and Payment.

The multiplicity is not sufficiently clear from the drawing to establish whether each order has exactly one payment, whether payment records can be repeated, or whether an order can exist before payment. The final diagram should show the intended rule explicitly.

5. Multiplicity in UML

Multiplicity describes how many instances of one class may be associated with an instance of another class.

Common multiplicity notations include:

- 1 : Exactly one instance.
- 0. .1 : zero or one instance.
- 0..* :zero or many instances.
- 1..* : one or many instances.

For example, if one customer can place several orders, the association may show 1 at the Customer end and 0..* at the Order end, assuming every order belongs to exactly one customer.

Multiplicities should reflect the intended business rules and should be consistent throughout the diagram.

6. UML Notation Used

The class diagram uses the following UML conventions:

- **Class name:** Identifies the class.
- **Attributes:** Describe the information stored by a class.
- **Operations:** Describe the actions that a class can perform.
- **Association lines:** Represent relationships between classes.
- **Multiplicity labels:** Specify the number of related instances allowed by a relationship.

Attributes and operations should be placed in their appropriate compartments within each class.

SQL MODULE YR2 CSDS

7. Design Observations

The current design identifies the main entities of the proposed online shopping system. Before final submission, the group should check the following points:

1. Ensure that all class names, attributes, and operations are readable.
2. Use consistent data type and naming conventions.
3. Verify the multiplicities on every association.
4. Clarify the multiplicity and meaning of the Order–Payment relationship.
5. Clarify how Cart relates to Product and Customer.
6. Check whether the duplicate-looking identifiers in Order are intentional.
7. Ensure that the electronic diagram, handwritten diagram, and report use the same design.

These observations are intended to help the group verify its own design; they do not change the original classes in the submitted diagram.

8. Conclusion

The UML class diagram for the Online Shopping System represents seven classes: Admin, Customer, Product, Category, Cart, Order, and Payment.

The classes contain attributes and operations that describe the principal information and activities of the proposed system. Their associations illustrate administrative product management, customer product viewing, order placement, shopping cart activities, product categorisation, and payment-related activities.

The diagram provides a structural model of the proposed system. Clearly specifying all relationships and multiplicities will make the design easier to understand and provide a stronger foundation for future implementation.

SQL MODULE YR2 CSDS